

Sprint 3 – Precificação Dinâmica com Integração Solar

Equipe

- Andrey Crence Fernandes – RM 573840
- Giulliana Maistro Brasolin – RM 569381
- Mikaella Mirela Dos Santos Lucindo – RM 573775
- Lara Dos Santos Cândido Alves – RM 573827
- Lucas Parkin Devito – RM 573251

1. Visão geral

O AsterCharge é um projeto para gerenciar eletropostos de forma inteligente. A ideia é ajustar o preço da recarga de acordo com a demanda e a disponibilidade de energia solar. Quando a demanda está alta, o preço aumenta. Quando há energia solar disponível e pouca demanda, o preço diminui para incentivar a recarga.

O projeto evoluiu em três etapas:

- **Sprint 1:** identificação dos problemas, proposta da solução e definição dos requisitos.
- **Sprint 2:** prova de conceito da precificação dinâmica em um Arduino Uno, simulado no Tinkercad, com LEDs e display LCD.
- **Sprint 3 (esta entrega):** o protótipo foi refeito com um ESP32 programado em MicroPython, simulado no Wokwi, e passou a considerar também a disponibilidade de energia solar e o horário de pico no cálculo do preço.
-

Simulação online: <https://wokwi.com/projects/475598428946468865>

2. O que o protótipo faz

A cada meio segundo, o ESP32 repete este ciclo:

1. Lê a luz no sensor LDR, que representa a disponibilidade de energia solar.
2. Lê o potenciômetro, que representa a demanda no eletroposto.
3. Verifica se o botão de horário de pico está pressionado.

4. Calcula o preço do kWh a partir desses três dados, partindo de um preço base de R\$ 1,00.
5. Acende o LED correspondente: verde para desconto, amarelo para preço normal e vermelho para acréscimo.
6. Mostra o estado, o preço e os níveis de solar e demanda no display LCD e no monitor serial.

3. Esquema de integração dos componentes

3.1 Diagrama de blocos

O protótipo tem três partes. Na entrada ficam os componentes que recebem as informações sobre luz, demanda e horário de pico. No centro, o ESP32 executa o programa em MicroPython e calcula o preço. Na saída ficam os três LEDs e o display LCD, que mostram o resultado. A Figura 1 mostra essa organização.

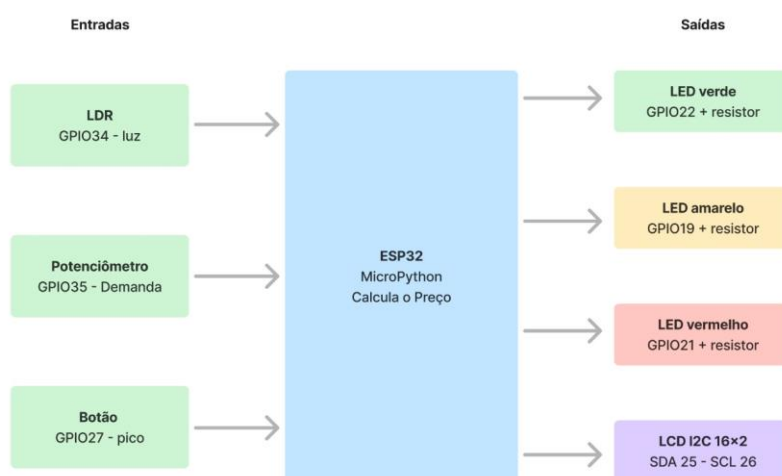


Figura 1 – Diagrama de blocos do protótipo (entradas, ESP32 e saídas)

A Tabela 1 detalha cada componente, o pino do ESP32 em que está ligado e o papel que cumpre.

Tabela 1 – Componentes, pinos e funções

Componente	Pino do ESP32	Função no protótipo
LDR (sensor de luz)	GPIO34	Entrada analógica. Representa a energia solar disponível (leitura de 0 a 4095).

Componente	Pino do ESP32	Função no protótipo
Potenciômetro	GPIO35	Entrada analógica. Representa a demanda no eletroposto (leitura de 0 a 4095).
Botão	GPIO27	Entrada digital com pull-up interno. Pressionado = horário de pico.
LED verde + resistor	GPIO22	Sinaliza desconto (preço abaixo de R\$ 1,00).
LED amarelo + resistor	GPIO19	Sinaliza preço normal.
LED vermelho + resistor	GPIO21	Sinaliza acréscimo (preço a partir de R\$ 1,50).
Display LCD 16×2 (I2C)	SDA: GPIO25 / SCL: GPIO26	Mostra estado, preço e níveis de solar e demanda (endereço I2C 0x27).

O código está dividido em dois arquivos. O main.py contém a lógica de leitura dos sensores, cálculo do preço e controle das saídas. Já o lcd_i2c.py é responsável pelo controle do display LCD usando a comunicação I2C. O circuito da simulação está montado no Wokwi e aparece na Figura 2.

3.2 Regras de cálculo do preço

O preço começa em R\$ 1,00 e recebe os seguintes ajustes, um após o outro:

- **Demanda (potenciômetro):** acima de 3000 é demanda alta e multiplica o preço por 1,40; acima de 1500 é demanda média e multiplica por 1,15; até 1500 é demanda baixa e não altera o preço.
- **Horário de pico (botão pressionado):** multiplica o preço por 1,30.
- **Luz solar (LDR):** acima de 3000 é solar alta e multiplica por 0,80, o que dá desconto; abaixo de 1000 é solar baixa e multiplica por 1,10; entre 1000 e 3000 não altera o preço.

Com o preço final calculado, o LED é escolhido assim:

- **Verde (desconto):** preço abaixo de R\$ 1,00.
- **Amarelo (normal):** preço de R\$ 1,00 até menos de R\$ 1,50.
- **Vermelho (acrécimo):** preço a partir de R\$ 1,50.

Dois exemplos: com demanda alta, horário de pico e pouca luz, o preço fica $1,00 \times 1,40 \times 1,30 \times 1,10 \approx \text{R\$ } 2,00$, e o LED vermelho acende. Com demanda baixa, fora do pico e muita luz, o preço fica $1,00 \times 0,80 = \text{R\$ } 0,80$, e o LED verde acende.

3.3 Fluxograma do programa

O programa começa configurando os pinos, o display LCD, a comunicação I2C e os LEDs. Depois, entra em um ciclo contínuo que lê os sensores, calcula o preço, aciona os LEDs e atualiza as informações exibidas. A Figura 2 mostra esse fluxo.

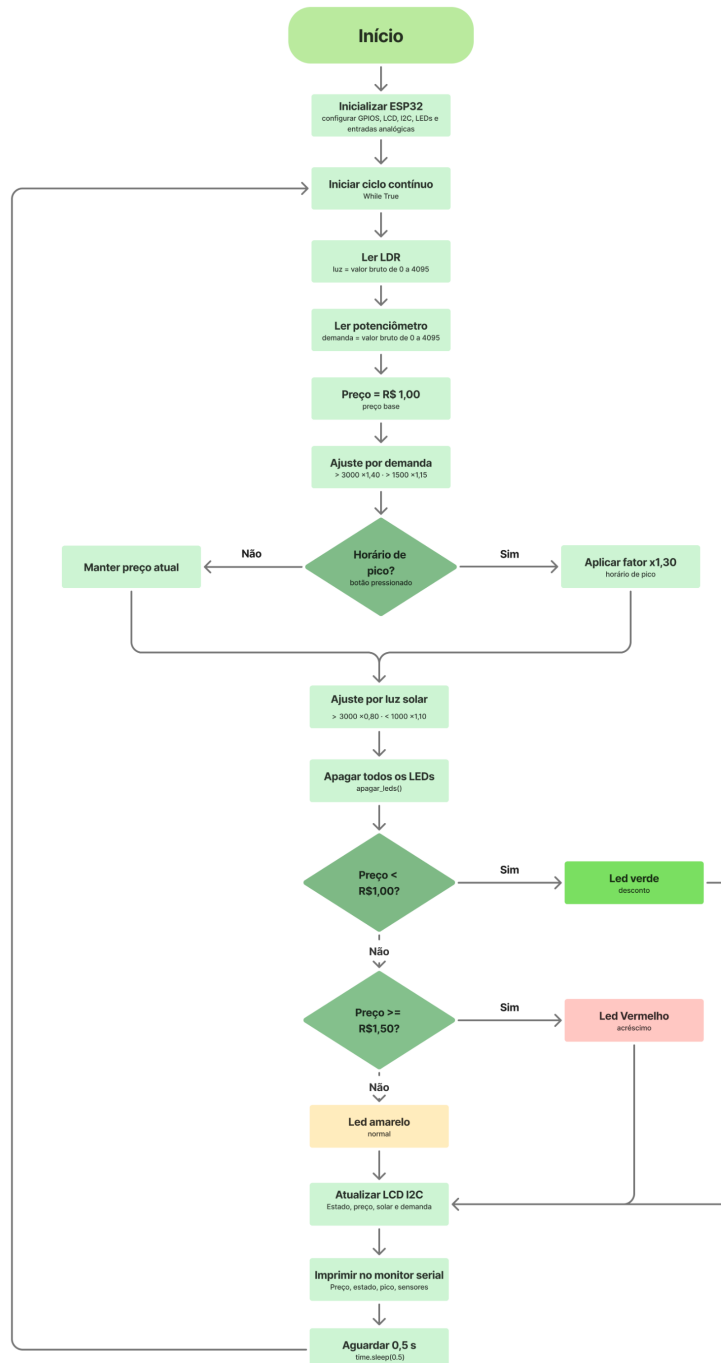


Figura 2 – Fluxograma do ciclo de funcionamento

4. Justificativa técnica das escolhas

As escolhas do protótipo seguem três critérios: funcionar bem na simulação, ser simples de explicar e ajustar, e poder ser usado em uma instalação real. Como é uma simulação, a luz, a demanda e o horário de pico são representados por componentes simples. O texto abaixo também mostra de onde essas informações poderiam vir em um eletroposto real

4.1 Por que o ESP32 no lugar do Arduino Uno

- **Conectividade:** o ESP32 tem Wi-Fi e Bluetooth integrados, o que permite, numa próxima etapa, enviar preço e dados de sessão para o aplicativo do AsterCharge. O Arduino Uno não tem esse recurso.
- **Leitura mais precisa:** o conversor analógico do ESP32 tem 12 bits (valores de 0 a 4095), contra 10 bits (0 a 1023) do Uno. Isso dá mais níveis para distinguir variações de luz e de demanda.
- **Mais capacidade:** o ESP32 tem processador mais rápido e mais memória, o que permite rodar MicroPython e deixa espaço para novas funções.
- **Adequação a IoT:** é uma placa barata e muito usada em projetos de internet das coisas, incluindo monitoramento de energia.

4.2 Por que MicroPython e Wokwi

O MicroPython usa Python, que é mais fácil de ler e manter do que C++. O Wokwi permite simular o ESP32, acompanhar o monitor serial e compartilhar o projeto por link, facilitando a avaliação e a demonstração em vídeo. Como o Tinkercad, usado na Sprint 2, não simula o ESP32, foi necessário trocar de simulador.

4.3 Por que esses sensores e entradas

- **LDR (energia solar):** o LDR muda sua resistência conforme a quantidade de luz. Ele é usado para simular a disponibilidade de energia solar, e a luminosidade pode ser ajustada no simulador. Em um eletroposto real, essa informação poderia vir do inversor solar ou de um medidor de irradiância.
- **Potenciômetro (demanda):** permite variar a demanda de baixa a alta e testar diferentes preços. Em uma instalação real, esse dado viria da quantidade de recargas ativas ou da potência utilizada.
- **Botão (horário de pico):** representa o horário de pico de forma simples, com uma opção de ligado e desligado. Em um eletroposto real, essa informação viria de um relógio com os horários definidos pela distribuidora de energia.

4.4 Por que essas saídas

- **LEDs verde, amarelo e vermelho:** seguem a lógica de semáforo, que qualquer usuário entende sem precisar ler. Verde indica desconto, amarelo indica preço normal e vermelho indica acréscimo.
- **Display LCD:** mostra o valor exato do preço e os níveis de solar e demanda, complementando a informação dos LEDs.
- **Monitor serial:** registra a cada ciclo os valores lidos e o preço calculado, o que permite conferir o funcionamento e coletar os dados dos testes.

4.5 Por que a lógica de multiplicadores

O preço começa com um valor base e recebe ajustes de acordo com a demanda, o horário de pico e a luz solar. Essa lógica é simples de explicar e fácil de alterar, pois basta mudar os valores no código. A demanda alta e o horário de pico aumentam o preço, enquanto a pouca luz solar aumenta um pouco a tarifa e a luz solar alta reduz o preço, incentivando o uso quando há energia renovável disponível.

5. Como executar e demonstrar o protótipo

O protótipo funciona de forma contínua no simulador Wokwi. Não é preciso instalar nada: basta abrir o link do projeto no navegador.

5.1 Passos para rodar:

1. Abrir o projeto no Wokwi: <https://wokwi.com/projects/475598428946468865>.
2. Iniciar a simulação no botão verde. O ESP32 começa a ler os sensores, e o display LCD passa a mostrar o estado e o preço.
3. Abrir o monitor serial do Wokwi, onde o ESP32 escreve os dados calculados a cada ciclo.
4. Alterar os sensores durante a simulação: clicar no LDR para mudar a luminosidade, mexer no potenciômetro para mudar a demanda e clicar no botão para simular o horário de pico. Enquanto o botão estiver pressionado, o sistema considera horário de pico.

A cada 0,5 segundo o sistema lê os valores de novo e atualiza os LEDs, o LCD e o monitor serial.

5.2 Cenários de demonstração

Os três cenários abaixo mostram as três respostas do sistema: desconto, tarifa normal e acréscimo.

- **Ação:** aumentar a luminosidade do LDR, manter a demanda baixa e deixar o botão sem pressionar.
- **Resultado:** há muita energia solar e pouca demanda, então o sistema reduz o valor da recarga para R\$ 0,80. O LED verde acende e o LCD mostra DESCONTO.

Cenário 2 – Tarifa normal

- **Ação:** ajustar o LDR e o potenciômetro para valores intermediários, com o botão sem pressionar.
- **Resultado:** geração solar e demanda estão equilibradas, então a tarifa fica normal. Com luz média e demanda média, o preço é R\$ 1,15. O LED amarelo acende e o LCD mostra NORMAL.

Cenário 3 – Acréscimo

- **Ação:** diminuir a luz do LDR, aumentar a demanda e pressionar o botão.
- **Resultado:** há pouca energia solar, demanda alta e horário de pico, condições que pressionam a rede elétrica. O sistema aplica acréscimo e o preço sobe para cerca de R\$ 2,00. O LED vermelho acende e o LCD mostra ACRESCIMO (sem acento, porque o display não exibe acentos).
-

5.3 Dados exibidos pelo protótipo

Durante os testes no Wokwi, o protótipo apresentou diferentes valores de tarifa de acordo com as condições de luz solar, demanda e horário de pico. As informações são exibidas no LCD e também podem ser acompanhadas pelo monitor serial.

Cenário	1	–	Desconto
Com alta disponibilidade de luz solar e baixa demanda, o sistema apresentou			

DESCONTO

R\$

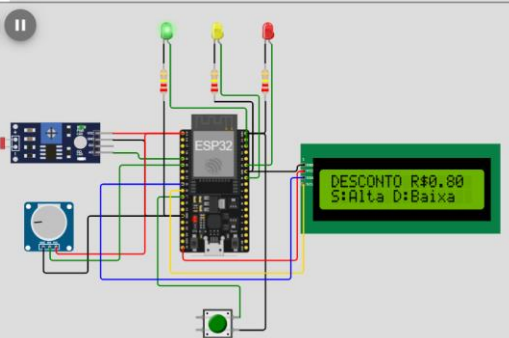
0,80

WOKWI SAVE SHARE

main.py diagram.json lcd_i2c.py

```
1 from machine import Pin, ADC, I2C
2 from lcd_i2c import LcdI2C
3 import time
4
5
6 i2c = I2C(0, scl=Pin(26), sda=Pin(25), freq=400000)
7 print("I2C:", [hex(a) for a in i2c.scan()])
8
9 lcd = LcdI2C(i2c, addr=0x27)
10 lcd.linha(0, "Iniciando...")
11
12
13 botao = Pin(27, Pin.IN, Pin.PULL_UP)
14
15 ldr = ADC(Pin(34))
16 ldr.atten(ADC.ATTN_11DB)
17
18 pot = ADC(Pin(35))
19 pot.atten(ADC.ATTN_11DB)
20
21
22 led_amarelo = Pin(19, Pin.OUT)
23 led_vermelho = Pin(21, Pin.OUT)
24 led_verde = Pin(22, Pin.OUT)
25
26 PRECO_BASE = 1.00
27
28
29 def apagar_leds():
30     led_vermelho.off()
31     led_amarelo.off()
32     led_verde.off()
33
34
```

Simulation



Horário Pico: False
Solar: Alta | LDR: 3645
Demanda: Baixa | Pot: 620

Preço: R\$ 0.8
Estado: DESCONTO
Horário Pico: False
Solar: Alta | LDR: 3645
Demanda: Baixa | Pot: 620

Cenário

2

-

Tarifa

normal

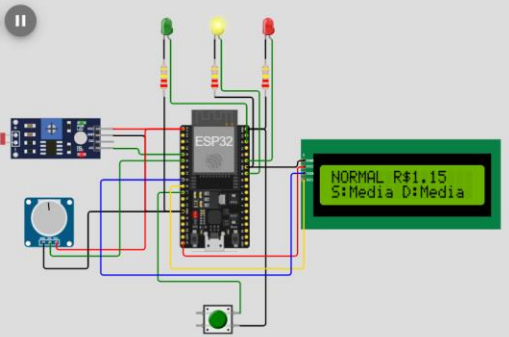
Com níveis médios de luz solar e demanda, fora do horário de pico, o sistema apresentou
NORMAL – R\$ 1,15.

WOKWI SAVE SHARE

main.py diagram.json lcd_i2c.py

```
1 from machine import Pin, ADC, I2C
2 from lcd_i2c import LcdI2C
3 import time
4
5
6 i2c = I2C(0, scl=Pin(26), sda=Pin(25), freq=400000)
7 print("I2C:", [hex(a) for a in i2c.scan()])
8
9 lcd = LcdI2C(i2c, addr=0x27)
10 lcd.linha(0, "Iniciando...")
11
12
13 botao = Pin(27, Pin.IN, Pin.PULL_UP)
14
15 ldr = ADC(Pin(34))
16 ldr.atten(ADC.ATTN_11DB)
17
18 pot = ADC(Pin(35))
19 pot.atten(ADC.ATTN_11DB)
20
21
22 led_amarelo = Pin(19, Pin.OUT)
23 led_vermelho = Pin(21, Pin.OUT)
24 led_verde = Pin(22, Pin.OUT)
25
26 PRECO_BASE = 1.00
27
28
29 def apagar_leds():
30     led_vermelho.off()
31     led_amarelo.off()
32     led_verde.off()
33
34
```

Simulation



Horário Pico: False
Solar: Media | LDR: 2045
Demanda: Media | Pot: 1957

Preço: R\$ 1.15
Estado: NORMAL
Horário Pico: False
Solar: Media | LDR: 2045
Demanda: Media | Pot: 1957

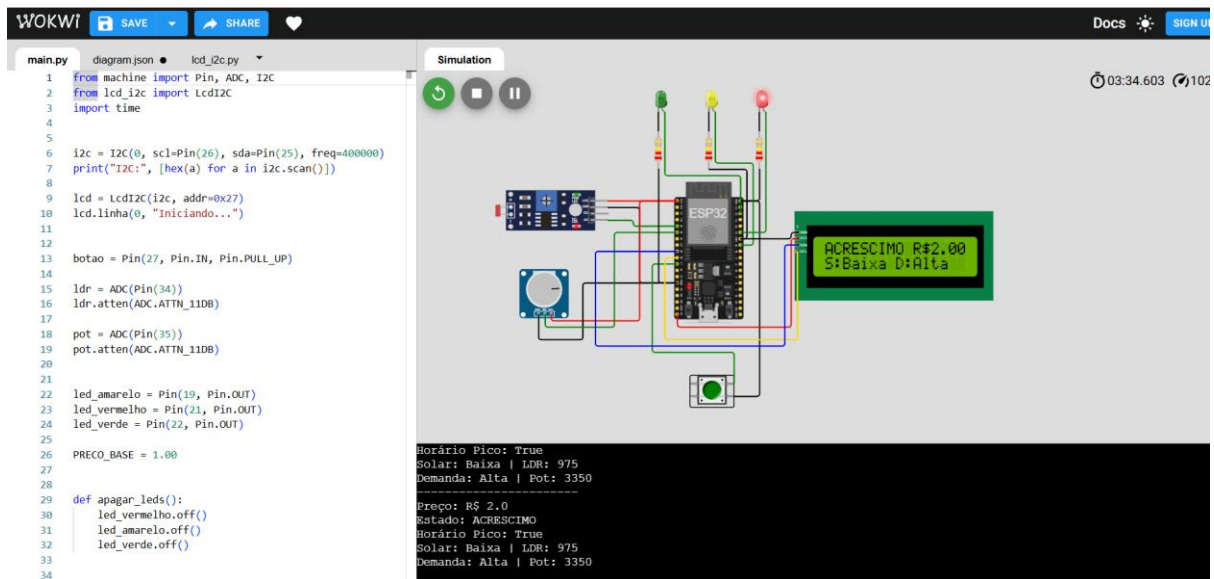
Cenário

3

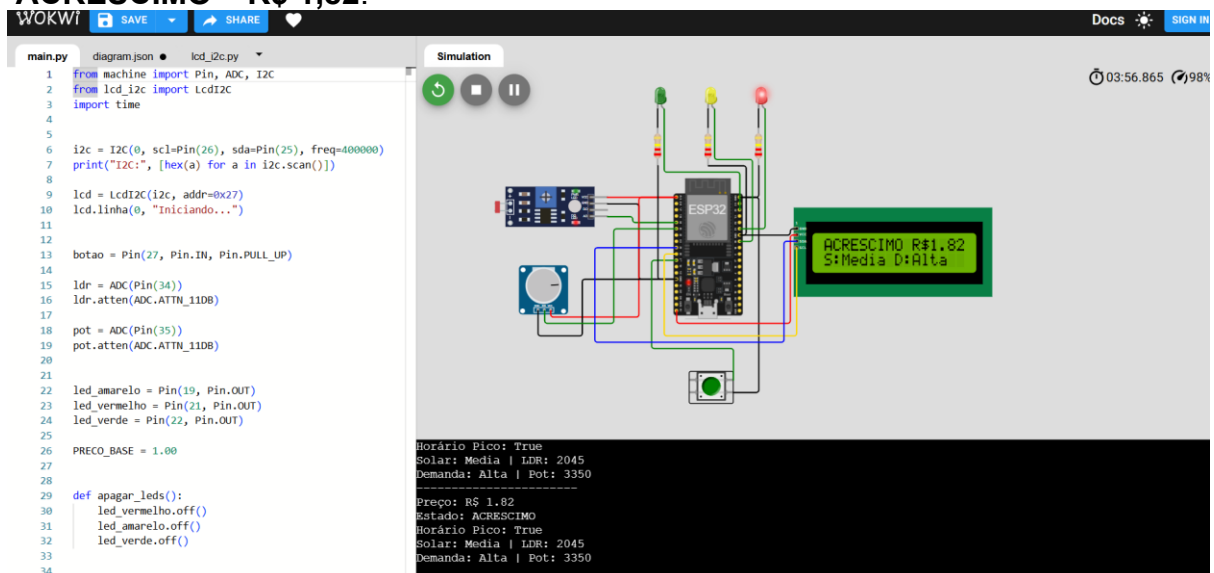
-

Acréscimo

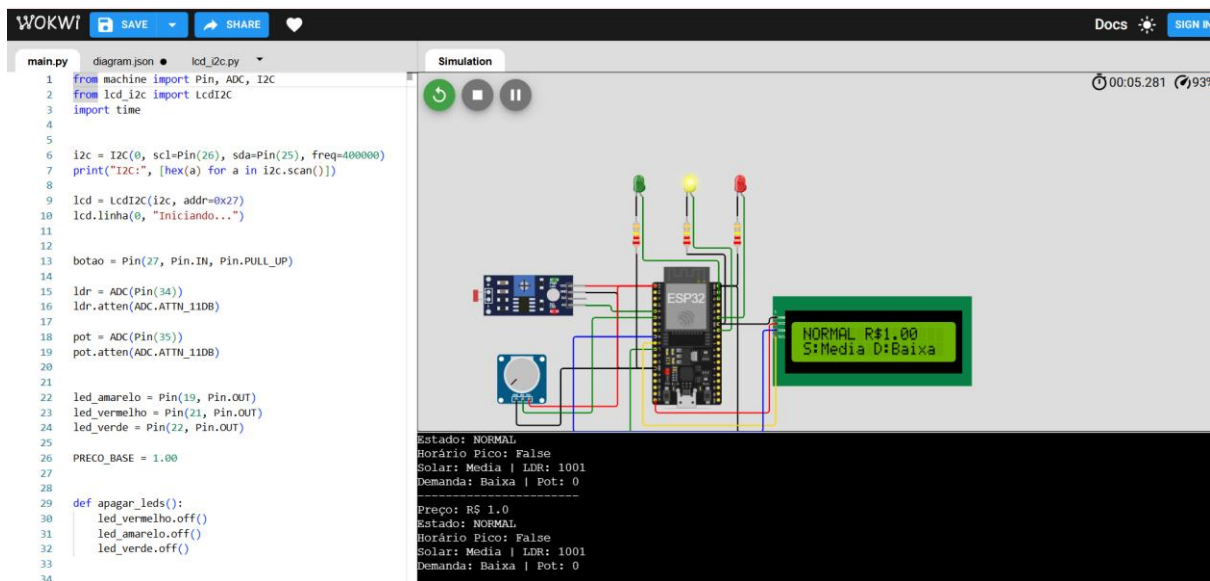
Com baixa disponibilidade de luz solar, demanda alta e horário de pico, o sistema apresentou
ACRÉSCIMO – R\$ 2,00.



Cenário 4 – Sem sol
Com demanda alta e horário de pico, o sistema apresentou **ACRÉSCIMO – R\$ 1,82.**



Cenário 5 – Teste adicional
Em outra condição de demanda baixa e nível médio de luz solar, o sistema apresentou **NORMAL – R\$ 1,00.**



6. Conexão com os conteúdos da disciplina

O protótipo aplica, em pequena escala, conceitos de Energias Renováveis e Sustentáveis. Esta seção mostra como o projeto se relaciona com os três pontos pedidos na Sprint 3: **sustentabilidade, automação inteligente e eficiência energética**:

- **Sustentabilidade** : A energia solar depende da luz do dia e pode variar ao longo do tempo. O protótipo representa essa variação usando o LDR: quando há muita luz, a tarifa diminui e incentiva a recarga quando há mais energia solar disponível. Quando há pouca luz e a demanda está alta, o preço aumenta e desestimula a recarga. Dessa forma, o sistema busca incentivar o uso de energia renovável e melhorar o aproveitamento da energia disponível.
- **Automação inteligente**: O sistema funciona sem intervenção humana. O ESP32 lê os sensores, calcula o preço com base nas regras definidas e controla os LEDs e o display, repetindo esse processo a cada meio segundo. Como o ESP32 possui Wi-Fi, o sistema pode futuramente enviar os preços e dados de recarga para o aplicativo do AsterCharge e receber informações reais de geração solar e consumo.
- **Eficiência energética** O preço dinâmico funciona como um sinal para incentivar o usuário a recarregar fora dos horários de pico e quando há maior disponibilidade de energia solar. Assim, a demanda fica mais distribuída ao longo do dia e a infraestrutura de recarga é melhor

aproveitada. Os LEDs e o display mostram essas informações de forma clara para o usuário.

6.4 Contribuição de cada tecnologia

- O **ESP32 com MicroPython** lê os sensores, calcula o preço e controla as saídas automaticamente.
- O **LDR** representa a disponibilidade de energia solar e ajuda a definir o preço da recarga.
- O **potenciômetro** representa a demanda e permite testar diferentes níveis de consumo.
- O **botão** representa o horário de pico e informa quando ele está ativo.
- Os **LEDs e o display LCD** mostram ao usuário o preço e a situação da recarga.
- A **simulação no Wokwi** permite testar o sistema antes de uma instalação real, sem precisar dos componentes físicos.

6.5 Conteúdos da disciplina que aparecem no protótipo

- **Geração solar e sua variabilidade:** o LDR representa a disponibilidade de energia solar, que muda conforme a luz.
- **Curva de carga e horário de pico:** o botão representa o horário de pico, período em que a demanda pela rede é maior.
- **Gestão pelo lado da demanda e tarifa horária:** o preço muda conforme demanda, horário e disponibilidade solar, em lógica semelhante à da tarifa horária praticada no Brasil, como a Tarifa Branca.
- **Mobilidade elétrica:** o preço calculado é o da recarga de veículos elétricos em um eletroposto.
- **Automação e IoT aplicadas à energia:** sensores, microcontrolador e atuadores trabalham de forma integrada e automática.